Andrea Colombo - Algorithms for Massive Datasets course project - A.A 2025/2026

Andrea Colombo

Abstract — This report contains the documentation related to the project submission for the *Algorithms for Massive Datasets* course. Taught by *Dario Malchiodi* at *Università degli studi di Milano.*
We present the implementation of two stream analysis algorithms: Flajolet-Martin and Alon-Matias-Szegedy, starting from the theory behind them and going all the way through the implementation choices and experimental results.

Table of Contents

1 Introduction

The analysis of massive datasets has become an increasingly important problem in the later years. These scenarios pose significant challenges due to the sheer amount of data to be proceseed; in most cases, storing the entire dataset in memory is not feasible.
Stream analysis algorithms address this issue by processing the dataset sequentially, using limited amounts of memory and providing estimations about the stream properties we are interested in. Since these algorithms do not need to store the entirety of the stream in memory, they are used across a moltitude of domains and hardware, ranging from huge server rigs to tiny embedded microcontrollers.

The algorithms and tasks we are going to explore are the following:

- Using the *Flajolet-Martin Algorithm* [1] to produce and estimation of the unique userIDs present in the dataset. The full description of the algorithm and the implementation choices can be found in Section 4.

1.1 Development Environment

This project was developed using *Google Colab* as the main computation environment. The Jupyter Notebook submitted alongside this project may require some modifications before being run in a local environment.
Below is a brief description of the computational resources available on the CoLab runtime and the versions of the main libraries used in this project:

- **CPU**: Intel Xeon CPU with 2 vCPUs (virtual CPUs) and 13GB of RAM.
- **Python Version**: 3.13.15 (built with GCC 11.4.0)
- **PySpark Version**: 4.0.4
- **xxHash Version**: 4.0.1

> Please note that these versions are correct at the time of writing: 2026-09-07

2 Dataset Description

The dataset used for this project is the **New York Times Articles & Comments (2020)** [2] dataset, freely available and licensed under the **CC-BY-NC-SA-4.0** license.

The datset, once downloaded, has a size of approximately **6.15 GBs** presents itself in the form of multiple files:

- **nyt-articles-2020.csv**: Contains all the articles published in 2020 by the NYT.
- **nyt-comments-2020.csv**: this file contains all the comments relative to the articles found in *nyt-articles-2020.csv*.
- **nyt-comments-part0.csv .. nyt-comments-part9.csv**: these fails simply contain the data found in *nyt-comments-2020.csv* split in 10 different partitions.
- **test.csv**: Not relevant for our use case

- **train.csv**: Not relevant for our use case

2.1 Preprocessing Techniques

During the preprocessing phase of the project we discarded all the columns that are not relevant for our use case, in particular:

- For the Flajolet-Martin portion of the project (Section 4), we discarded all the columns except the *UserID* and, since all the fields inside the schema are tagged as **nullable**, we excluded all the null entries to avoid using garbage data.

2.2 Subsampling

In order to ensure a reasonable execution time, we introduced a method that allows the user to load only a part of the dataset instead of the whole. This method can be tweaked by modifying the **SAMPLING_PROPORTION** (see: Section 3) variable inside the notebook provided alongside this document.

3 System Configuration

The python notebook submitted with this project can be configured with the following set of variables:

- **ENABLE_LOGGING**: enables additional prints during the notebook execution
- **SAMPLING_PROPORTION**: How much of the whole dataset we want to use for the current run, valid if in range $(0, 1]$.
- **FM_NUM_HASHES**: Number of hash functions to use for the Flajolet-Martin implementation.

4 Flajolet–Martin Algorithm

First introduced in 1985 [1], the Flajolet-Martin algorithm is a probabilistic algorithm that aims at estimating the number of distinct elements through the use of hash functions.
The core idea is to leverage two very important properties of hash functions:

- **Determinism**: the hash function always maps the same value to the same result
- **Uniform Distribution**: the hashed values are uniformly distributed over the binary space

But why do we focus on the number of trailing zeros (tail length) of the hashed value?
The probability that a single hash value ends with n trailing zeros is $\frac{1}{2^n}$.
From this forumula we can derive that:

- The probability of a single hash value not having n trailing zeros is $1 - \frac{1}{2^n} = 1 - 2^{-n}$
- The probability of **none** of k hash values having n trailing zeros is $\left(1 - \frac{1}{2^n}\right)^k$. This can be approximated to $e^{-k2^{-n}}$ for very large numbers.

Therefore, if the maximum number of trailing zeros is $R_{\max}$, it implies that we have seen approximatively $2^{R_{\max}}$ unique elements so far inside the stream.

4.1 The Outlier Problem

Using a single hash function has one big issue, since we are taking the maximum number of trailing zeros from a single source, we are susceptible to outliers.

The solution is simply using multiple hash functions and compute the median of their results. This has been verified to provide more accurate results but it still has one problem, it always results in estimates that are power of 2.
In order to fix this issue, we compute the average of the median estimations we just computed.

4.2 Space/Time Complexity

This algorithm has a time complexity of $O(kn)$ where n is the length of the stream and k is the constant value that refers to the computational cost of the hash functions and update procedure. The space complexity is $O(k)$ instead, we only need to store a constant amount of information needed to update the trailing zeros counts; the precise memory usage depends on how many hash function we decide to use for the algorithm.

4.3 Implementation Details

4.3.1 Hash Function Choice

The hash function used in the submitted implementation is xxhash, an extremely fast non cryptographic hashing algorithm. In particular, this implementation uses the xxh3 64 bits version, xxhash is one of the most used hashing algorithms for non cryptographic uses in real world use cases (PySpark uses it too).

4.3.2 Algorithm implementation

The portion of the algorithm responsible for computing the tail lengths and keeping track of the maximum values can be described with 3 simple functions. Each partition yields a sequence of length *FM_NUM_HASHES* that is then used to compute the final estimation.

python

```python
def ctz(x):
    if x == 0:
        return 64
    return (x & -x).bit_length() - 1

def hash64bits(value, seed):
    return xxhash.xxh64(value.to_bytes(8, "little"), seed=seed).intdigest()

def fm_partition(it):
    registers = [0] * FM_NUM_HASHES
    for user in it:
        for i in range(FM_NUM_HASHES):
            bits = hash64bits(user, i)
            r = ctz(bits)
```

```
            if r > registers[i]:
                registers[i] = r

    yield registers
```

4.4 Experimental Results

5 AMS Algorithm

5.1 Space/Time Complexity

5.2 Implementation Details

5.3 Experimental Results

6 Bloom Filter

A bloom filter [3] is a probabilistic hash based data structure that is used for checking whether or not an element may be in the dataset while also being ok with having some false positives.
In this project we implemented this data structure to test it against the stream of unique *userIDs* of the people who left at least one comment on articles of the *Opinion* category (Section 2). More specifically, we started from the set of unique *userIDs* of the stream described above, we then fabricated n_{fake} new unique identifiers that are not present in the initial stream to use them to test for false positive rates at different configurations (more details in Section 6.3).

The bloom filter is structured as follows:

- An array of n bits is used to store information about the elements seen inside a stream or a generic dataset.
- k hash functions are used to map an element to k bit positions.
  - If the bit array already has all those bits set to one, then we may have already seen the element. We are still susceptible to false positives because, since the bit array size is finite, multiple hashes (mapped with the modulo inside the bit array) may end up resulting in the same bit positions being used.
  - If **at least one** of those bit positions is not 1, then the element was never encountered before (there can be no false negatives).

It is clear that the performance of the filter itself depends heavily on the number of bits used for the array and the number of hash functions used on every element. These parameters heavily depend on the use case and the available resources for the filter.

6.1 False Positives Theory

To understand why the bloom filter is used, we must also go into the math behind the number of false positives.
For the rest of this section, we are going to assume m as the number of bits and k as the number of hash functions used in the filter.
We are also going to assume that the selected hash functions map each element uniformely over the m bits.

- After a single element insertion, the probability that a specific bit is not going to be set to 1 as a result of the k hash functions is:
  - $P(\text{bit remains 0}) = \left(1 - \frac{1}{m}\right)^k$
- Given the probabilities described in the first point, for n insertions, the probability that a bit is not going to be set to 1 is:
  - $P(\text{bit remains 0 after n}) = \left(1 - \frac{1}{m}\right)^{kn}$
- The probability of a false positive then becomes:
  - $P(\text{false positive}) = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k$
- We can now approximate $\left(1 - \frac{1}{m}\right)^{kn}$ to $e^{\frac{-kn}{m}}$ to obtain:
  - $P(\text{false positive approx}) = \left(1 - e^{\frac{-kn}{m}}\right)^k$

We can see that, the higher m and k are, the less likely the filter is to report a false positive when it's being used.
It goes without saying that we cannot simply keep driving these numbers up without encountering memory and time issues. To correctly make use of a bloom filter, the user must choose m based on their memory constraints and k depending on the time that is allocatable to the task of computing the hash functions for a given element.

If we want to minimize the probability of false positives, we can choose the ideal number of hash functions k using the following equation and rounding the result to the closest integer:

- $k = \frac{m}{n} \ln(2)$

Where $\frac{m}{n}$ is the number of individual bits allocated for each element. This means that, even without knowing the exact value of n, we can use an estimation of it to choose a number of hash functions that is close to the ideal one.

This assumes the estimation is not too different from the actual value

6.2 Space/Time Complexity

When it comes to time complexity, both checking for an element and inserting a new one is equal to $O(k)$, where k is the number of hash functions used in the filter implementation. The time performance of a bloom filter varies greatly depending on the complexity of the hash functions used (which is generally tied to the complexity of the type of the elements to hash). Generalising this to a stream it becomes trivial that, for n elements, the complexity goes up to $O(nk)$. The

space complexity of the bloom filter is trivial too, it is equal to $O(m)$ where m is the number of bits used.

6.3 Implementation Details

Implementing a bloom filter from scratch in python is very straight forward. Since python's integers are not restricted to a maximum amount of bits, we can use a single int as our bit array, this means that accessing individual bits can be done with simple bitwise operations that are very efficient. The class is required to have at least the following methods:

In order to use stable hash functions, we can store them in the object state as an array of k elements, the hash function chosen for this implementation is taken from the xxhash library.

Below is a code snippet to showcase the membership check and insertion methods for the described class:

```python
def add(self, x):
    self.bits |= self._map_bits(x)

def _map_bits(self, x):
    mask = 0
    for hf in self.hash_fns:
        mask |= (1 << (hf(x) % self.nbits))
    return mask

def contains(self, x):
    xmask = self._map_bits(x)
    return (self.bits & xmask) == xmask
```

- self.nbits: number of bits we want to store for the filter (m).
- self.hash_fns: array of k hash functions that return a 64 bit integer.
- self.bits: bit array that keeps track of the filter state (implemented using python's unbounded ints).

6.4 Chosen Task

As introduced in Section 6, we are going to test the proposed implementation in the following way:

- We first gather the set of all unique *UserIDs* of users that commented at least one article belonging to the *Opinion* section.
- We then generate a set of fake *UserIDs* that are guaranteed not to be found inside the first set.
- We "bootstrap" the filter by adding all the *UserIDs* of the first set to it.
- We iterate over the second set to calculate the false positive rate with different configurations of m and k.

The workflow we just described is going to allow us to compare the empirical results gathered with the theory explained in Section 6.1

6.5 Experimental Results

6.6 Scaling to Massive Dataset

The bloom filter proposed in this project is suited for scaling to massive datasets, given that it is configured with m and k values that are approriate for the task at hand.
Once the filter has been configured, the memory consumption is fixed at m bits, regarless of the number of elements inserted into the filter. Moreover, as seen in Section 6.2, the time complexity depends on the value of k; since this value is usually constant, the time complexity can be seen as $O(1)$ for each insertion and membership check.

On the other hand, approaches that use set-like data structures to keep track of the elements have a space complexity of $O(n)$. Additionally, as the number of elements inside a set grows, the performance of the set operations tends to degrade as well due to hash collisions (both on open and closed hashing approaches).

A bloom filter therefore provides a userful accuracy/memory tradeoff for large scale data processing. When the expected number of elements is known, the filter can be configured with $m = bn$ bits, where b is the number of bits allocated per element. In this case, the memory consumption also grows linearly with the number of elements, but the required memory can be determined in advance and does not depend on the size of the individual elements.

7 Plagiarism and AI Usage Statement

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.

Bibliography

[1] F. Philippe and M. G. Nigel, "Proba bit istic Cou nting Algorithms for Data Base Applications," *Journal of computer and system sciences 31*, 1985.

[2] B. Dornel, "New York Times Articles & Comments (2020)." 2021.

[3] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970, doi: 10.1145/362686.362692.